

Chapter 2. Installing Kafka

This chapter describes how to get started with the Apache Kafka broker, including how to set up Apache ZooKeeper, which is used by Kafka for storing metadata for the brokers. The chapter will also cover basic configuration options for Kafka deployments, as well as some suggestions for selecting the correct hardware to run the brokers on. Finally, we cover how to install multiple Kafka brokers as part of a single cluster and things you should know when using Kafka in a production environment.

Environment Setup

Before using Apache Kafka, your environment needs to be set up with a few prerequisites to ensure it runs properly. The following sections will guide you through that process.

Choosing an Operating System

Apache Kafka is a Java application and can run on many operating systems. While Kafka is capable of being run on many OSs, including Windows, macOS, Linux, and others, Linux is the recommended OS for the general use case. The installation steps in this chapter will focus on setting up and using Kafka in a Linux environment. For information on installing Kafka on Windows and macOS, see [Appendix A](#).

Installing Java

Prior to installing either ZooKeeper or Kafka, you will need a Java environment set up and functioning. Kafka and ZooKeeper work well with all OpenJDK-based Java implementations, including Oracle JDK. The latest versions of Kafka support both Java 8 and Java 11. The exact version installed can be the version provided by your OS or one directly downloaded from the web—for example, [the Oracle website for the Oracle version](#). Though ZooKeeper and Kafka will work with a runtime edition of Java, it is recommended when developing tools and applications to have the full Java Development Kit (JDK). It is recommended to install the latest released patch version of your Java environment, as older versions may have security vulnerabilities. The installation steps will assume you have installed JDK version 11 update 10 deployed at `/usr/java/jdk-11.0.10`.

Installing ZooKeeper

Apache Kafka uses Apache ZooKeeper to store metadata about the Kafka cluster, as well as consumer client details, as shown in [Figure 2-1](#). ZooKeeper is a centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services. This book won't go into extensive detail about ZooKeeper but

will limit explanations to only what is needed to operate Kafka. While it is possible to run a ZooKeeper server using scripts contained in the Kafka distribution, it is trivial to install a full version of ZooKeeper from the distribution.

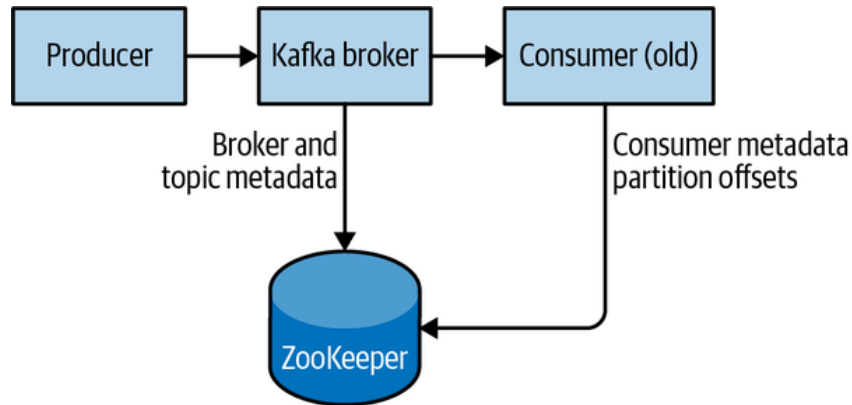


Figure 2-1. Kafka and ZooKeeper

Kafka has been tested extensively with the stable 3.5 release of ZooKeeper and is regularly updated to include the latest release. In this book, we will be using ZooKeeper 3.5.9, which can be downloaded from the [ZooKeeper website](#).

Standalone server

ZooKeeper comes with a base example config file that will work well for most use cases in `/usr/local/zookeeper/config/zoo_sample.cfg`. However, we will manually create ours with some basic settings for demo purposes in this book. The following example installs ZooKeeper with a basic configuration in `/usr/local/zookeeper`, storing its data in `/var/lib/zookeeper`:

```
# tar -zxf apache-zookeeper-3.5.9-bin.tar.gz
# mv apache-zookeeper-3.5.9-bin /usr/local/zookeeper
# mkdir -p /var/lib/zookeeper
# cp > /usr/local/zookeeper/conf/zoo.cfg << EOF
> tickTime=2000
> dataDir=/var/lib/zookeeper
> clientPort=2181
> EOF
# export JAVA_HOME=/usr/java/jdk-11.0.10
# /usr/local/zookeeper/bin/zkServer.sh start
JMX enabled by default
Using config: /usr/local/zookeeper/bin/./conf/zoo.cfg
Starting zookeeper ... STARTED
#
```

You can now validate that ZooKeeper is running correctly in standalone mode by connecting to the client port and sending the four-letter command `srvr`. This will return basic ZooKeeper information from the running server:

```
# telnet localhost 2181
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
srvr
```

```
Zookeeper version: 3.5.9-83df9301aa5c2a5d284a9940177808c01bc35cef, built on 01/06/
Latency min/avg/max: 0/0/0
Received: 1
Sent: 0
Connections: 1
Outstanding: 0
Zxid: 0x0
Mode: standalone
Node count: 5
Connection closed by foreign host.
#
```

ZooKeeper ensemble

ZooKeeper is designed to work as a cluster, called an *ensemble*, to ensure high availability. Due to the balancing algorithm used, it is recommended that ensembles contain an odd number of servers (e.g., 3, 5, and so on) as a majority of ensemble members (a *quorum*) must be working in order for ZooKeeper to respond to requests. This means that in a three-node ensemble, you can run with one node missing. With a five-node ensemble, you can run with two nodes missing.

SIZING YOUR ZOOKEEPER ENSEMBLE

Consider running ZooKeeper in a five-node ensemble. To make configuration changes to the ensemble, including swapping a node, you will need to reload nodes one at a time. If your ensemble cannot tolerate more than one node being down, doing maintenance work introduces additional risk. It is also not recommended to run more than seven nodes, as performance can start to degrade due to the nature of the consensus protocol.

Additionally, if you feel that five or seven nodes aren't supporting the load due to too many client connections, consider adding additional observer nodes for help in balancing read-only traffic.

To configure ZooKeeper servers in an ensemble, they must have a common configuration that lists all servers, and each server needs a *myid* file in the data directory that specifies the ID number of the server. If the hostnames of the servers in the ensemble are `zoo1.example.com`, `zoo2.example.com`, and `zoo3.example.com`, the configuration file might look like this:

```
tickTime=2000
dataDir=/var/lib/zookeeper
clientPort=2181
initLimit=20
syncLimit=5
server.1=zoo1.example.com:2888:3888
server.2=zoo2.example.com:2888:3888
server.3=zoo3.example.com:2888:3888
```

In this configuration, the `initLimit` is the amount of time to allow followers to connect with a leader. The `syncLimit` value limits how long out-of-sync followers can be with the leader. Both values are a number of `tickTime` units, which makes the `initLimit` $20 \times 2,000$ ms, or 40 sec-

onds. The configuration also lists each server in the ensemble. The servers are specified in the format *server.X=hostname:peerPort:leaderPort*, with the following parameters:

X

The ID number of the server. This must be an integer, but it does not need to be zero-based or sequential.

hostname

The hostname or IP address of the server.

peerPort

The TCP port over which servers in the ensemble communicate with one another.

leaderPort

The TCP port over which leader election is performed.

Clients only need to be able to connect to the ensemble over the *clientPort*, but the members of the ensemble must be able to communicate with one another over all three ports.

In addition to the shared configuration file, each server must have a file in the *dataDir* directory with the name *myid*. This file must contain the ID number of the server, which must match the configuration file. Once these steps are complete, the servers will start up and communicate with one another in an ensemble.

TESTING ZOOKEEPER ENSEMBLE ON A SINGLE MACHINE

It is possible to test and run a ZooKeeper ensemble on a single machine by specifying all hostnames in the config as `localhost` and have unique ports specified for *peerPort* and *leaderPort* for each instance. Additionally, a separate *zoo.cfg* would need to be created for each instance with a unique *dataDir* and *clientPort* defined for each instance. This can be useful for testing purposes only, but it is *not* recommended for production systems.

Installing a Kafka Broker

Once Java and ZooKeeper are configured, you are ready to install Apache Kafka. The current release can be downloaded from the [Kafka website](#). At press time, that version is 2.8.0 running under Scala version 2.13.0. The examples in this chapters are shown using version 2.7.0.

The following example installs Kafka in */usr/local/kafka*, configured to use the ZooKeeper server started previously and to store the message log segments stored in */tmp/kafka-logs*:

```
# tar -zxf kafka_2.13-2.7.0.tgz
# mv kafka_2.13-2.7.0 /usr/local/kafka
# mkdir /tmp/kafka-logs
```

```
# export JAVA_HOME=/usr/java/jdk-11.0.10
# /usr/local/kafka/bin/kafka-server-start.sh -daemon
/usr/local/kafka/config/server.properties
#
```

Once the Kafka broker is started, we can verify that it is working by performing some simple operations against the cluster: creating a test topic, producing some messages, and consuming the same messages.

Create and verify a topic:

```
# /usr/local/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --create
--replication-factor 1 --partitions 1 --topic test
Created topic "test".
# /usr/local/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092
--describe --topic test
Topic:test      PartitionCount:1      ReplicationFactor:1      Configs:
      Topic: test      Partition: 0      Leader: 0      Replicas: 0      Isr: 0
#
```

Produce messages to a test topic (use Ctrl-C to stop the producer at any time):

```
# /usr/local/kafka/bin/kafka-console-producer.sh --bootstrap-server
localhost:9092 --topic test
Test Message 1
Test Message 2
^C
#
```

Consume messages from a test topic:

```
# /usr/local/kafka/bin/kafka-console-consumer.sh --bootstrap-server
localhost:9092 --topic test --from-beginning
Test Message 1
Test Message 2
^C
Processed a total of 2 messages
#
```

DEPRECATION OF ZOOKEEPER CONNECTIONS ON KAFKA CLI UTILITIES

If you are familiar with older versions of the Kafka utilities, you may be used to using a `--zookeeper` connection string. This has been deprecated in almost all cases. The current best practice is to use the newer `--bootstrap-server` option and connect directly to the Kafka broker. If you are running in a cluster, you can provide the host:port of any broker in the cluster.

Configuring the Broker

The example configuration provided with the Kafka distribution is sufficient to run a standalone server as a proof of concept, but most likely will not be sufficient for large installations. There are numerous configura-

tion options for Kafka that control all aspects of setup and tuning. Most of the options can be left at the default settings, though, as they deal with tuning aspects of the Kafka broker that will not be applicable until you have a specific use case that requires adjusting these settings.

General Broker Parameters

There are several broker configuration parameters that should be reviewed when deploying Kafka for any environment other than a stand-alone broker on a single server. These parameters deal with the basic configuration of the broker, and most of them must be changed to run properly in a cluster with other brokers.

broker.id

Every Kafka broker must have an integer identifier, which is set using the `broker.id` configuration. By default, this integer is set to `0`, but it can be any value. It is essential that the integer must be unique for each broker within a single Kafka cluster. The selection of this number is technically arbitrary, and it can be moved between brokers if necessary for maintenance tasks. However, it is highly recommended to set this value to something intrinsic to the host so that when performing maintenance it is not onerous to map broker ID numbers to hosts. For example, if your hostnames contain a unique number (such as `host1.example.com`, `host2.example.com`, etc.), then `1` and `2` would be good choices for the `broker.id` values, respectively.

listeners

Older versions of Kafka used a simple `port` configuration. This can still be used as a backup for simple configurations but is a deprecated config. The example configuration file starts Kafka with a listener on TCP port `9092`. The new `listeners` config is a comma-separated list of URIs that we listen on with the listener names. If the listener name is not a common security protocol, then another config

`listener.security.protocol.map` must also be configured. A listener is defined as `<protocol>://<hostname>:<port>`. An example of a legal listener config is

`PLAINTEXT://localhost:9092,SSL://:9091`. Specifying the hostname as `0.0.0.0` will bind to all interfaces. Leaving the hostname empty will bind it to the default interface. Keep in mind that if a port lower than `1024` is chosen, Kafka must be started as root. Running Kafka as root is not a recommended configuration.

zookeeper.connect

The location of the ZooKeeper used for storing the broker metadata is set using the `zookeeper.connect` configuration parameter. The example configuration uses a ZooKeeper running on port `2181` on the local host, which is specified as `localhost:2181`. The format for this parameter is a semicolon-separated list of `hostname:port/path` strings, which include:

hostname

The hostname or IP address of the ZooKeeper server.

port

The client port number for the server.

/path

An optional ZooKeeper path to use as a chroot environment for the Kafka cluster. If it is omitted, the root path is used.

If a chroot path (a path designated to act as the root directory for a given application) is specified and does not exist, it will be created by the broker when it starts up.

WHY USE A CHROOT PATH?

It is generally considered to be good practice to use a chroot path for the Kafka cluster. This allows the ZooKeeper ensemble to be shared with other applications, including other Kafka clusters, without a conflict. It is also best to specify multiple ZooKeeper servers (which are all part of the same ensemble) in this configuration. This allows the Kafka broker to connect to another member of the ZooKeeper ensemble in the event of server failure.

log.dirs

Kafka persists all messages to disk, and these log segments are stored in the directory specified in the `log.dir` configuration. For multiple directories, the config `log.dirs` is preferable. If this value is not set, it will default back to `log.dir`. `log.dirs` is a comma-separated list of paths on the local system. If more than one path is specified, the broker will store partitions on them in a “least-used” fashion, with one partition’s log segments stored within the same path. Note that the broker will place a new partition in the path that has the least number of partitions currently stored in it, not the least amount of disk space used, so an even distribution of data across multiple directories is not guaranteed.

num.recovery.threads.per.data.dir

Kafka uses a configurable pool of threads for handling log segments. Currently, this thread pool is used:

- When starting normally, to open each partition’s log segments
- When starting after a failure, to check and truncate each partition’s log segments
- When shutting down, to cleanly close log segments

By default, only one thread per log directory is used. As these threads are only used during startup and shutdown, it is reasonable to set a larger number of threads in order to parallelize operations. Specifically, when recovering from an unclean shutdown, this can mean the difference of several hours when restarting a broker with a large number of partitions! When setting this parameter, remember that the number configured is per log directory specified with `log.dirs`. This means that if `num`.

`recovery.threads.per.data.dir` is set to 8, and there are 3 paths specified in `log.dirs`, this is a total of 24 threads.

auto.create.topics.enable

The default Kafka configuration specifies that the broker should automatically create a topic under the following circumstances:

- When a producer starts writing messages to the topic
- When a consumer starts reading messages from the topic
- When any client requests metadata for the topic

In many situations, this can be undesirable behavior, especially as there is no way to validate the existence of a topic through the Kafka protocol without causing it to be created. If you are managing topic creation explicitly, whether manually or through a provisioning system, you can set the `auto.create.topics.enable` configuration to `false`.

auto.leader.rebalance.enable

In order to ensure a Kafka cluster doesn't become unbalanced by having all topic leadership on one broker, this config can be specified to ensure leadership is balanced as much as possible. It enables a background thread that checks the distribution of partitions at regular intervals (this interval is configurable via `leader.imbalance.check.interval.seconds`).

If leadership imbalance exceeds another config, `leader.imbalance.per.broker.percentage`, then a rebalance of preferred leaders for partitions is started.

delete.topic.enable

Depending on your environment and data retention guidelines, you may wish to lock down a cluster to prevent arbitrary deletions of topics. Disabling topic deletion can be set by setting this flag to `false`.

Topic Defaults

The Kafka server configuration specifies many default configurations for topics that are created. Several of these parameters, including partition counts and message retention, can be set per topic using the administrative tools (covered in [Chapter 12](#)). The defaults in the server configuration should be set to baseline values that are appropriate for the majority of the topics in the cluster.

USING PER-TOPIC OVERRIDES

In older versions of Kafka, it was possible to specify per-topic overrides for these configurations in the broker configuration using the parameters

`log.retention.hours.per.topic`, `log.retention.bytes.per.topic`, and `log.segment.bytes.per.topic`. These parameters are no longer supported, and overrides must be specified using the administrative tools.

num.partitions

The `num.partitions` parameter determines how many partitions a new topic is created with, primarily when automatic topic creation is enabled (which is the default setting). This parameter defaults to one partition. Keep in mind that the number of partitions for a topic can only be increased, never decreased. This means that if a topic needs to have fewer partitions than `num.partitions`, care will need to be taken to manually create the topic (discussed in [Chapter 12](#)).

As described in [Chapter 1](#), partitions are the way a topic is scaled within a Kafka cluster, which makes it important to use partition counts that will balance the message load across the entire cluster as brokers are added. Many users will have the partition count for a topic be equal to, or a multiple of, the number of brokers in the cluster. This allows the partitions to be evenly distributed to the brokers, which will evenly distribute the message load. For example, a topic with 10 partitions operating in a Kafka cluster with 10 hosts with leadership balanced among all 10 hosts will have optimal throughput. This is not a requirement, however, as you can also balance message load in other ways, such as having multiple topics.

There are several factors to consider when choosing the number of partitions:

- What is the throughput you expect to achieve for the topic? For example, do you expect to write 100 KBps or 1 GBps?
- What is the maximum throughput you expect to achieve when consuming from a single partition? A partition will always be consumed completely by a single consumer (even when not using consumer groups, the consumer must read all messages in the partition). If you know that your slower consumer writes the data to a database and this database never handles more than 50 MBps from each thread writing to it, then you know you are limited to 50 MBps throughput when consuming from a partition.
- You can go through the same exercise to estimate the maximum throughput per producer for a single partition, but since producers are typically much faster than consumers, it is usually safe to skip this.
- If you are sending messages to partitions based on keys, adding partitions later can be very challenging, so calculate throughput based on your expected future usage, not the current usage.
- Consider the number of partitions you will place on each broker and available disk space and network bandwidth per broker.
- Avoid overestimating, as each partition uses memory and other resources on the broker and will increase the time for metadata updates and leadership transfers.
- Will you be mirroring data? You may need to consider the throughput of your mirroring configuration as well. Large partitions can become a bottleneck in many mirroring configurations.
- If you are using cloud services, do you have IOPS (input/output operations per second) limitations on your VMs or disks? There may be hard caps on the number of IOPS allowed depending on your cloud service and VM configuration that will cause you to hit quotas. Having too many partitions can have the side effect of increasing the amount of IOPS due to the parallelism involved.

With all this in mind, it's clear that you want many partitions, but not too many. If you have some estimate regarding the target throughput of the topic and the expected throughput of the consumers, you can divide the target throughput by the expected consumer throughput and derive the number of partitions this way. So if we want to be able to write and read 1 GBps from a topic, and we know each consumer can only process 50 MBps, then we know we need at least 20 partitions. This way, we can have 20 consumers reading from the topic and achieve 1 GBps.

If you don't have this detailed information, our experience suggests that limiting the size of the partition on the disk to less than 6 GB per day of retention often gives satisfactory results. Starting small and expanding as needed is easier than starting too large.

default.replication.factor

If auto-topic creation is enabled, this configuration sets what the replication factor should be for new topics. Replication strategy can vary depending on the desired durability or availability of a cluster and will be discussed more in later chapters. The following is a brief recommendation if you are running Kafka in a cluster that will prevent outages due to factors outside of Kafka's internal capabilities, such as hardware failures.

It is highly recommended to set the replication factor to at least 1 above the `min.insync.replicas` setting. For more fault-resistant settings, if you have large enough clusters and enough hardware, setting your replication factor to 2 above the `min.insync.replicas` (abbreviated as RF++) can be preferable. RF++ will allow easier maintenance and prevent outages. The reasoning behind this recommendation is to allow for one planned outage within the replica set and one unplanned outage to occur simultaneously. For a typical cluster, this would mean you'd have a minimum of three replicas of every partition. An example of this is if there is a network switch outage, disk failure, or some other unplanned problem during a rolling deployment or upgrade of Kafka or the underlying OS, you can be assured there will still be an additional replica available. This will be discussed more in [Chapter 7](#).

log.retention.ms

The most common configuration for how long Kafka will retain messages is by time. The default is specified in the configuration file using the `log.retention.hours` parameter, and it is set to 168 hours, or one week. However, there are two other parameters allowed, `log.retention.minutes` and `log.retention.ms`. All three of these control the same goal (the amount of time after which messages may be deleted), but the recommended parameter to use is `log.retention.ms`, as the smaller unit size will take precedence if more than one is specified. This will ensure that the value set for `log.retention.ms` is always the one used. If more than one is specified, the smaller unit size will take precedence.

RETENTION BY TIME AND LAST MODIFIED TIMES

Retention by time is performed by examining the last modified time (mtime) on each log segment file on disk. Under normal cluster operations, this is the time that the log segment was closed, and represents the timestamp of the last message in the file. However, when using administrative tools to move partitions between brokers, this time is not accurate and will result in excess retention for these partitions. For more information on this, see [Chapter 12](#) discussing partition moves.

log.retention.bytes

Another way to expire messages is based on the total number of bytes of messages retained. This value is set using the `log.retention.bytes` parameter, and it is applied per partition. This means that if you have a topic with 8 partitions, and `log.retention.bytes` is set to 1 GB, the amount of data retained for the topic will be 8 GB at most. Note that all

retention is performed for individual partitions, not the topic. This means that should the number of partitions for a topic be expanded, the retention will also increase if `log.retention.bytes` is used. Setting the value to `-1` will allow for infinite retention.

CONFIGURING RETENTION BY SIZE AND TIME

If you have specified a value for both `log.retention.bytes` and `log.retention.ms` (or another parameter for retention by time), messages may be removed when either criteria is met. For example, if `log.retention.ms` is set to 86400000 (1 day) and `log.retention.bytes` is set to 1000000000 (1 GB), it is possible for messages that are less than 1 day old to get deleted if the total volume of messages over the course of the day is greater than 1 GB. Conversely, if the volume is less than 1 GB, messages can be deleted after 1 day even if the total size of the partition is less than 1 GB. It is recommended, for simplicity, to choose either size- or time-based retention—and not both—to prevent surprises and unwanted data loss, but both can be used for more advanced configurations.

log.segment.bytes

The log retention settings previously mentioned operate on log segments, not individual messages. As messages are produced to the Kafka broker, they are appended to the current log segment for the partition. Once the log segment has reached the size specified by the `log.segment.bytes` parameter, which defaults to 1 GB, the log segment is closed and a new one is opened. Once a log segment has been closed, it can be considered for expiration. A smaller log segment size means that files must be closed and allocated more often, which reduces the overall efficiency of disk writes.

Adjusting the size of the log segments can be important if topics have a low produce rate. For example, if a topic receives only 100 megabytes per day of messages, and `log.segment.bytes` is set to the default, it will take 10 days to fill one segment. As messages cannot be expired until the log segment is closed, if `log.retention.ms` is set to 604800000 (1 week), there will actually be up to 17 days of messages retained until the closed log segment expires. This is because once the log segment is closed with the current 10 days of messages, that log segment must be retained for 7 days before it expires based on the time policy (as the segment cannot be removed until the last message in the segment can be expired).

RETRIEVING OFFSETS BY TIMESTAMP

The size of the log segment also affects the behavior of fetching offsets by timestamp. When requesting offsets for a partition at a specific timestamp, Kafka finds the log segment file that was being written at that time. It does this by using the creation and last modified time of the file, and looking for a file that was created before the timestamp specified and last modified after the timestamp. The offset at the beginning of that log segment (which is also the filename) is returned in the response.

log.roll.ms

Another way to control when log segments are closed is by using the `log.roll.ms` parameter, which specifies the amount of time after which a log segment should be closed. As with the `log.retention.bytes` and `log.retention.ms` parameters, `log.segment.bytes` and `log.roll.ms` are not mutually exclusive properties. Kafka will close a log segment either when the size limit is reached or when the time limit is reached, whichever comes first. By default, there is no setting for `log.roll.ms`, which results in only closing log segments by size.

DISK PERFORMANCE WHEN USING TIME-BASED SEGMENTS

When using a time-based log segment limit, it is important to consider the impact on disk performance when multiple log segments are closed simultaneously. This can happen when there are many partitions that never reach the size limit for log segments, as the clock for the time limit will start when the broker starts and will always execute at the same time for these low-volume partitions.

min.insync.replicas

When configuring your cluster for data durability, setting `min.insync.replicas` to 2 ensures that at least two replicas are caught up and “in sync” with the producer. This is used in tandem with setting the producer config to ack “all” requests. This will ensure that at least two replicas (leader and one other) acknowledge a write for it to be successful. This can prevent data loss in scenarios where the leader acks a write, then suffers a failure and leadership is transferred to a replica that does not have a successful write. Without these durable settings, the producer would think it successfully produced, and the message(s) would be dropped on the floor and lost. However, configuring for higher durability has the side effect of being less efficient due to the extra overhead involved, so clusters with high-throughput that can tolerate occasional message loss aren’t recommended to change this setting from the default of 1. See [Chapter 7](#) for more information.

message.max.bytes

The Kafka broker limits the maximum size of a message that can be produced, configured by the `message.max.bytes` parameter, which defaults to 1000000, or 1 MB. A producer that tries to send a message larger than this will receive an error back from the broker, and the message will not be accepted. As with all byte sizes specified on the broker, this configuration deals with compressed message size, which means that producers can send messages that are much larger than this value uncompressed, provided they compress to under the configured `message.max.bytes` size.

There are noticeable performance impacts from increasing the allowable message size. Larger messages will mean that the broker threads that deal with processing network connections and requests will be working longer on each request. Larger messages also increase the size of disk

writes, which will impact I/O throughput. Other storage solutions, such as blob stores and/or tiered storage, may be another method of addressing large disk write issues, but will not be covered in this chapter.

COORDINATING MESSAGE SIZE CONFIGURATIONS

The message size configured on the Kafka broker must be coordinated with the `fetch.message.max.bytes` configuration on consumer clients. If this value is smaller than `message.max.bytes`, then consumers that encounter larger messages will fail to fetch those messages, resulting in a situation where the consumer gets stuck and cannot proceed. The same rule applies to the `replica.fetch.max.bytes` configuration on the brokers when configured in a cluster.

Selecting Hardware

Selecting an appropriate hardware configuration for a Kafka broker can be more art than science. Kafka itself has no strict requirement on a specific hardware configuration and will run without issue on most systems. Once performance becomes a concern, however, there are several factors that can contribute to the overall performance bottlenecks: disk throughput and capacity, memory, networking, and CPU. When scaling Kafka very large, there can also be constraints on the number of partitions that a single broker can handle due to the amount of metadata that needs to be updated. Once you have determined which performance types are the most critical for your environment, you can select an optimized hardware configuration appropriate for your budget.

Disk Throughput

The performance of producer clients will be most directly influenced by the throughput of the broker disk that is used for storing log segments. Kafka messages must be committed to local storage when they are produced, and most clients will wait until at least one broker has confirmed that messages have been committed before considering the send successful. This means that faster disk writes will equal lower produce latency.

The obvious decision when it comes to disk throughput is whether to use traditional spinning hard disk drives (HDDs) or solid-state disks (SSDs). SSDs have drastically lower seek and access times and will provide the best performance. HDDs, on the other hand, are more economical and provide more capacity per unit. You can also improve the performance of HDDs by using more of them in a broker, whether by having multiple data directories or by setting up the drives in a redundant array of independent disks (RAID) configuration. Other factors, such as the specific drive technology (e.g., serial attached storage or serial ATA), as well as the quality of the drive controller, will affect throughput. Generally, observations show that HDD drives are typically more useful for clusters with very high storage needs but aren't accessed as often, while SSDs are better options if there is a very large number of client connections.

Disk Capacity

Capacity is the other side of the storage discussion. The amount of disk capacity that is needed is determined by how many messages need to be retained at any time. If the broker is expected to receive 1 TB of traffic each day, with 7 days of retention, then the broker will need a minimum of 7 TB of usable storage for log segments. You should also factor in at least 10% overhead for other files, in addition to any buffer that you wish to maintain for fluctuations in traffic or growth over time.

Storage capacity is one of the factors to consider when sizing a Kafka cluster and determining when to expand it. The total traffic for a cluster can be balanced across the cluster by having multiple partitions per topic, which will allow additional brokers to augment the available capacity if the density on a single broker will not suffice. The decision on how much disk capacity is needed will also be informed by the replication strategy chosen for the cluster (which is discussed in more detail in [Chapter 7](#)).

Memory

The normal mode of operation for a Kafka consumer is reading from the end of the partitions, where the consumer is caught up and lagging behind the producers very little, if at all. In this situation, the messages the consumer is reading are optimally stored in the system's page cache, resulting in faster reads than if the broker has to reread the messages from disk. Therefore, having more memory available to the system for page cache will improve the performance of consumer clients.

Kafka itself does not need much heap memory configured for the Java Virtual Machine (JVM). Even a broker that is handling 150,000 messages per second and a data rate of 200 megabits per second can run with a 5 GB heap. The rest of the system memory will be used by the page cache and will benefit Kafka by allowing the system to cache log segments in use. This is the main reason it is not recommended to have Kafka colocated on a system with any other significant application, as it will have to share the use of the page cache. This will decrease the consumer performance for Kafka.

Networking

The available network throughput will specify the maximum amount of traffic that Kafka can handle. This can be a governing factor, combined with disk storage, for cluster sizing. This is complicated by the inherent imbalance between inbound and outbound network usage that is created by Kafka's support for multiple consumers. A producer may write 1 MB per second for a given topic, but there could be any number of consumers that create a multiplier on the outbound network usage. Other operations, such as cluster replication (covered in [Chapter 7](#)) and mirroring (discussed in [Chapter 10](#)), will also increase requirements. Should the network interface become saturated, it is not uncommon for cluster replication to fall behind, which can leave the cluster in a vulnerable state. To prevent the network from being a major governing factor, it is recom-

mended to run with at least 10 Gb NICs (Network Interface Cards). Older machines with 1 Gb NICs are easily saturated and aren't recommended.

CPU

Processing power is not as important as disk and memory until you begin to scale Kafka very large, but it will affect overall performance of the broker to some extent. Ideally, clients should compress messages to optimize network and disk usage. The Kafka broker must decompress all message batches, however, in order to validate the checksum of the individual messages and assign offsets. It then needs to recompress the message batch in order to store it on disk. This is where most of Kafka's requirement for processing power comes from. This should not be the primary factor in selecting hardware, however, unless clusters become very large with hundreds of nodes and millions of partitions in a single cluster. At that point, selecting more performant CPU can help reduce cluster sizes.

Kafka in the Cloud

In recent years, a more common installation for Kafka is within cloud computing environments, such as Microsoft Azure, Amazon's AWS, or Google Cloud Platform. There are many options to have Kafka set up in the cloud and managed for you via vendors like Confluent or even through Azure's own Kafka on HDInsight, but the following is some simple advice if you plan to manage your own Kafka clusters manually. In most cloud environments, you have a selection of many compute instances, each with a different combination of CPU, memory, IOPS, and disk. The various performance characteristics of Kafka must be prioritized in order to select the correct instance configuration to use.

Microsoft Azure

In Azure, you can manage the disks separately from the virtual machine (VM), so deciding your storage needs does not need to be related to the VM type selected. That being said, a good place to start on decisions is with the amount of data retention required, followed by the performance needed from the producers. If very low latency is necessary, I/O optimized instances utilizing premium SSD storage might be required. Otherwise, managed storage options (such as the Azure Managed Disks or the Azure Blob Storage) might be sufficient.

In real terms, experience in Azure shows that `Standard D16s v3` instance types are a good choice for smaller clusters and are performant enough for most use cases. To match high performant hardware and CPU needs, `D64s v4` instances have good performance that can scale for larger clusters. It is recommended to build out your cluster in an Azure availability set and balance partitions across Azure compute fault domains to ensure availability. Once you have a VM picked out, deciding on storage types can come next. It is highly recommended to use Azure Managed Disks rather than ephemeral disks. If a VM is moved, you run the risk of losing all the data on your Kafka broker. HDD Managed Disks are relatively inexpensive but do not have clearly defined SLAs from Microsoft on

availability. Premium SSDs or Ultra SSD configurations are much more expensive but are much quicker and are well supported with 99.99% SLAs from Microsoft. Alternatively, using Microsoft Blob Storage is an option if you are not as latency sensitive.

Amazon Web Services

In AWS, if very low latency is necessary, I/O optimized instances that have local SSD storage might be required. Otherwise, ephemeral storage (such as the Amazon Elastic Block Store) might be sufficient.

A common choice in AWS is either the `m4` or `r3` instance types. The `m4` will allow for greater retention periods, but the throughput to the disk will be less because it is on elastic block storage. The `r3` instance will have much better throughput with local SSD drives, but those drives will limit the amount of data that can be retained. For the best of both worlds, it may be necessary to move up to either the `i2` or `d2` instance types, but they are significantly more expensive.

Configuring Kafka Clusters

A single Kafka broker works well for local development work, or for a proof-of-concept system, but there are significant benefits to having multiple brokers configured as a cluster, as shown in [Figure 2-2](#). The biggest benefit is the ability to scale the load across multiple servers. A close second is using replication to guard against data loss due to single system failures. Replication will also allow for performing maintenance work on Kafka or the underlying systems while still maintaining availability for clients. This section focuses on the steps to configure a Kafka basic cluster. [Chapter 7](#) contains more information on replication of data and durability.

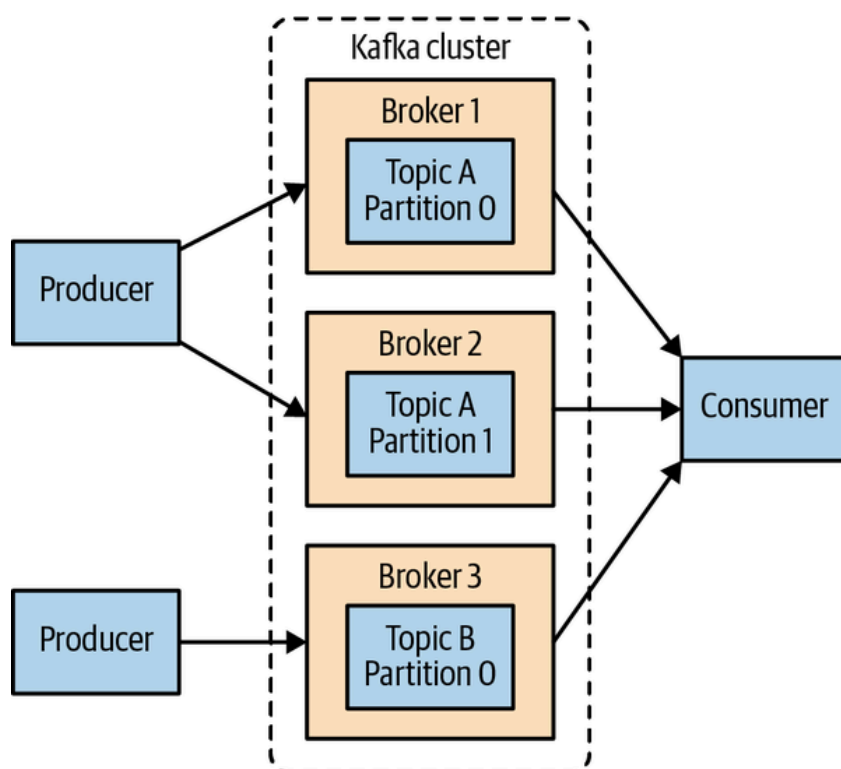


Figure 2-2. A simple Kafka cluster

How Many Brokers?

The appropriate size for a Kafka cluster is determined by several factors. Typically, the size of your cluster will be bound on the following key areas:

- Disk capacity
- Replica capacity per broker
- CPU capacity
- Network capacity

The first factor to consider is how much disk capacity is required for retaining messages and how much storage is available on a single broker. If the cluster is required to retain 10 TB of data and a single broker can store 2 TB, then the minimum cluster size is 5 brokers. In addition, increasing the replication factor will increase the storage requirements by at least 100%, depending on the replication factor setting chosen (see [Chapter 7](#)). Replicas in this case refer to the number of different brokers a single partition is copied to. This means that this same cluster, configured with a replication of 2, now needs to contain at least 10 brokers.

The other factor to consider is the capacity of the cluster to handle requests. This can exhibit through the other three bottlenecks mentioned earlier.

If you have a 10-broker Kafka cluster but have over 1 million replicas (i.e., 500,000 partitions with a replication factor of 2) in your cluster, each broker is taking on approximately 100,000 replicas in an evenly balanced scenario. This can lead to bottlenecks in the produce, consume, and controller queues. In the past, official recommendations have been to have no more than 4,000 partition replicas per broker and no more than 200,000 partition replicas per cluster. However, advances in cluster efficiency have allowed Kafka to scale much larger. Currently, in a well-configured environment, it is recommended to not have more than 14,000 partition replicas per broker and 1 million *replicas* per cluster.

As previously mentioned in this chapter, CPU usually is not a major bottleneck for most use cases, but it can be if there is an excessive amount of client connections and requests on a broker. Keeping an eye on overall CPU usage based on how many unique clients and consumer groups there are, and expanding to meet those needs, can help to ensure better overall performance in large clusters. Speaking to network capacity, it is important to keep in mind the capacity of the network interfaces and whether they can handle the client traffic if there are multiple consumers of the data or if the traffic is not consistent over the retention period of the data (e.g., bursts of traffic during peak times). If the network interface on a single broker is used to 80% capacity at peak, and there are two consumers of that data, the consumers will not be able to keep up with peak traffic unless there are two brokers. If replication is being used in the cluster, this is an additional consumer of the data that must be taken into account. You may also want to scale out to more brokers in a cluster in order to handle performance concerns caused by lesser disk throughput or system memory available.

Broker Configuration

There are only two requirements in the broker configuration to allow multiple Kafka brokers to join a single cluster. The first is that all brokers must have the same configuration for the `zookeeper.connect` parameter. This specifies the ZooKeeper ensemble and path where the cluster stores metadata. The second requirement is that all brokers in the cluster must have a unique value for the `broker.id` parameter. If two brokers attempt to join the same cluster with the same `broker.id`, the second broker will log an error and fail to start. There are other configuration parameters used when running a cluster—specifically, parameters that control replication, which are covered in later chapters.

OS Tuning

While most Linux distributions have an out-of-the-box configuration for the kernel-tuning parameters that will work fairly well for most applications, there are a few changes that can be made for a Kafka broker that will improve performance. These primarily revolve around the virtual memory and networking subsystems, as well as specific concerns for the disk mount point used for storing log segments. These parameters are typically configured in the `/etc/sysctl.conf` file, but you should refer to your Linux distribution documentation for specific details regarding how to adjust the kernel configuration.

Virtual memory

In general, the Linux virtual memory system will automatically adjust itself for the system workload. We can make some adjustments to how swap space is handled, as well as to dirty memory pages, to tune these for Kafka's workload.

As with most applications, specifically ones where throughput is a concern, it is best to avoid swapping at (almost) all costs. The cost incurred by having pages of memory swapped to disk will show up as a noticeable impact on all aspects of performance in Kafka. In addition, Kafka makes heavy use of the system page cache, and if the VM system is swapping to disk, there is not enough memory being allocated to page cache.

One way to avoid swapping is simply not to configure any swap space at all. Having swap is not a requirement, but it does provide a safety net if something catastrophic happens on the system. Having swap can prevent the OS from abruptly killing a process due to an out-of-memory condition. For this reason, the recommendation is to set the `vm.swappiness` parameter to a very low value, such as 1. The parameter is a percentage of how likely the VM subsystem is to use swap space rather than dropping pages from the page cache. It is preferable to reduce the amount of memory available for the page cache rather than utilize any amount of swap memory.

Previously, the recommendation for `vm.swappiness` was always to set it to 0. This value used to mean “do not swap unless there is an out-of-memory condition.” However, the meaning of this value changed as of Linux kernel version 3.5-rc1, and that change was backported into many distributions, including Red Hat Enterprise Linux kernels as of version 2.6.32-303. This changed the meaning of the value 0 to “never swap under any circumstances.” This is why a value of 1 is now recommended.

There is also a benefit to adjusting how the kernel handles dirty pages that must be flushed to disk. Kafka relies on disk I/O performance to provide good response times to producers. This is also the reason that the log segments are usually put on a fast disk, whether that is an individual disk with a fast response time (e.g., SSD) or a disk subsystem with significant NVRAM for caching (e.g., RAID). The result is that the number of dirty pages that are allowed, before the flush background process starts writing them to disk, can be reduced. Do this by setting the `vm.dirty_background_ratio` value lower than the default of 10. The value is a percentage of the total amount of system memory, and setting this value to 5 is appropriate in many situations. This setting should not be set to zero, however, as that would cause the kernel to continually flush pages, which would then eliminate the ability of the kernel to buffer disk writes against temporary spikes in the underlying device performance.

The total number of dirty pages allowed before the kernel forces synchronous operations to flush them to disk can also be increased by changing the value of `vm.dirty_ratio` to above the default of 20 (also a percentage of total system memory). There is a wide range of possible values for this setting, but between 60 and 80 is a reasonable number. This setting does introduce a small amount of risk, both in regard to the amount of unflushed disk activity as well as the potential for long I/O pauses if synchronous flushes are forced. If a higher setting for `vm.dirty_ratio` is chosen, it is highly recommended that replication be used in the Kafka cluster to guard against system failures.

When choosing values for these parameters, it is wise to review the number of dirty pages over time while the Kafka cluster is running under load, whether in production or simulated. The current number of dirty pages can be determined by checking the `/proc/vmstat` file:

```
# cat /proc/vmstat | egrep "dirty|writeback"
nr_dirty 21845
nr_writeback 0
nr_writeback_temp 0
nr_dirty_threshold 32715981
nr_dirty_background_threshold 2726331
#
```

Kafka uses file descriptors for log segments and open connections. If a broker has a lot of partitions, then that broker needs at least $(number_of_partitions) \times (partition_size/segment_size)$ to track all the log segments in addition to the number of connections the broker makes. As such, it is

recommended to update the `vm.max_map_count` to a very large number based on the above calculation. Depending on the environment, changing this value to 400,000 or 600,000 has generally been successful. It is also recommended to set `vm.overcommit_memory` to 0. Setting the default value of 0 indicates that the kernel determines the amount of free memory from an application. If the property is set to a value other than zero, it could lead the operating system to grab too much memory, depriving memory for Kafka to operate optimally. This is common for applications with high ingestion rates.

Disk

Outside of selecting the disk device hardware, as well as the configuration of RAID if it is used, the choice of filesystem for this disk can have the next largest impact on performance. There are many different filesystems available, but the most common choices for local filesystems are either Ext4 (fourth extended filesystem) or Extents File System (XFS). XFS has become the default filesystem for many Linux distributions, and this is for good reason: it outperforms Ext4 for most workloads with minimal tuning required. Ext4 can perform well but requires using tuning parameters that are considered less safe. This includes setting the commit interval to a longer time than the default of five to force less frequent flushes. Ext4 also introduced delayed allocation of blocks, which brings with it a greater chance of data loss and filesystem corruption in case of a system failure. The XFS filesystem also uses a delayed allocation algorithm, but it is generally safer than the one used by Ext4. XFS also has better performance for Kafka's workload without requiring tuning beyond the automatic tuning performed by the filesystem. It is also more efficient when batching disk writes, all of which combine to give better overall I/O throughput.

Regardless of which filesystem is chosen for the mount that holds the log segments, it is advisable to set the `noatime` mount option for the mount point. File metadata contains three timestamps: creation time (`ctime`), last modified time (`mtime`), and last access time (`atime`). By default, the `atime` is updated every time a file is read. This generates a large number of disk writes. The `atime` attribute is generally considered to be of little use, unless an application needs to know if a file has been accessed since it was last modified (in which case the `relatime` option can be used). The `atime` is not used by Kafka at all, so disabling it is safe. Setting `noatime` on the mount will prevent these timestamp updates from happening but will not affect the proper handling of the `ctime` and `mtime` attributes. Using the option `largeio` can also help improve efficiency for Kafka for when there are larger disk writes.

Networking

Adjusting the default tuning of the Linux networking stack is common for any application that generates a high amount of network traffic, as the kernel is not tuned by default for large, high-speed data transfers. In fact, the recommended changes for Kafka are the same as those suggested for most web servers and other networking applications. The first adjustment is to change the default and maximum amount of memory allocated

for the send and receive buffers for each socket. This will significantly increase performance for large transfers. The relevant parameters for the send and receive buffer default size per socket are `net.core.wmem_default` and `net.core.rmem_default`, and a reasonable setting for these parameters is 131072, or 128 KiB. The parameters for the send and receive buffer maximum sizes are `net.core.wmem_max` and `net.core.rmem_max`, and a reasonable setting is 2097152, or 2 MiB. Keep in mind that the maximum size does not indicate that every socket will have this much buffer space allocated; it only allows up to that much if needed.

In addition to the socket settings, the send and receive buffer sizes for TCP sockets must be set separately using the `net.ipv4.tcp_wmem` and `net.ipv4.tcp_rmem` parameters. These are set using three space-separated integers that specify the minimum, default, and maximum sizes, respectively. The maximum size cannot be larger than the values specified for all sockets using `net.core.wmem_max` and `net.core.rmem_max`. An example setting for each of these parameters is “4096 65536 2048000,” which is a 4 KiB minimum, 64 KiB default, and 2 MiB maximum buffer. Based on the actual workload of your Kafka brokers, you may want to increase the maximum sizes to allow for greater buffering of the network connections.

There are several other network tuning parameters that are useful to set. Enabling TCP window scaling by setting `net.ipv4.tcp_window_scaling` to 1 will allow clients to transfer data more efficiently, and allow that data to be buffered on the broker side. Increasing the value of `net.ipv4.tcp_max_syn_backlog` above the default of 1024 will allow a greater number of simultaneous connections to be accepted. Increasing the value of `net.core.netdev_max_backlog` to greater than the default of 1000 can assist with bursts of network traffic, specifically when using multigigabit network connection speeds, by allowing more packets to be queued for the kernel to process them.

Production Concerns

Once you are ready to move your Kafka environment out of testing and into your production operations, there are a few more things to think about that will assist with setting up a reliable messaging service.

Garbage Collector Options

Tuning the Java garbage-collection options for an application has always been something of an art, requiring detailed information about how the application uses memory and a significant amount of observation and trial and error. Thankfully, this has changed with Java 7 and the introduction of the Garbage-First garbage collector (G1GC). While G1GC was considered unstable initially, it saw marked improvement in JDK8 and JDK11. It is now recommended for Kafka to use G1GC as the default garbage collector. G1GC is designed to automatically adjust to different workloads and provide consistent pause times for garbage collection over the lifetime of the application. It also handles large heap sizes with ease by seg-

menting the heap into smaller zones and not collecting over the entire heap in each pause.

G1GC does all of this with a minimal amount of configuration in normal operation. There are two configuration options for G1GC used to adjust its performance:

MaxGCPauseMillis

This option specifies the preferred pause time for each garbage-collection cycle. It is not a fixed maximum—G1GC can and will exceed this time if required. This value defaults to 200 milliseconds. This means that G1GC will attempt to schedule the frequency of garbage collector cycles, as well as the number of zones that are collected in each cycle, such that each cycle will take approximately 200 ms.

InitiatingHeapOccupancyPercent

This option specifies the percentage of the total heap that may be in use before G1GC will start a collection cycle. The default value is 45. This means that G1GC will not start a collection cycle until after 45% of the heap is in use. This includes both the new (Eden) and old zone usage, in total.

The Kafka broker is fairly efficient with the way it utilizes heap memory and creates garbage objects, so it is possible to set these options lower. The garbage collector tuning options provided in this section have been found to be appropriate for a server with 64 GB of memory, running Kafka in a 5 GB heap. For `MaxGCPauseMillis`, this broker can be configured with a value of 20 ms. The value for `InitiatingHeapOccupancyPercent` is set to 35, which causes garbage collection to run slightly earlier than with the default value.

Kafka was originally released before the G1GC collector was available and considered stable. Therefore, Kafka defaults to using concurrent mark and sweep garbage collection to ensure compatibility with all JVMs. New best practice is to use G1GC for anything for Java 1.8 and later. The change is easy to make via environment variables. Using the `start` command from earlier in the chapter, modify it as follows:

```
# export KAFKA_JVM_PERFORMANCE_OPTS="-server -Xmx6g -Xms6g
-XX:MetaspaceSize=96m -XX:+UseG1GC
-XX:MaxGCPauseMillis=20 -XX:InitiatingHeapOccupancyPercent=35
-XX:G1HeapRegionSize=16M -XX:MinMetaspaceFreeRatio=50
-XX:MaxMetaspaceFreeRatio=80 -XX:+ExplicitGCInvokesConcurrent"
# /usr/local/kafka/bin/kafka-server-start.sh -daemon
/usr/local/kafka/config/server.properties
#
```

Datacenter Layout

For testing and development environments, the physical location of the Kafka brokers within a datacenter is not as much of a concern, as there is not as severe an impact if the cluster is partially or completely unavailable for short periods of time. However, when serving production traffic,

downtime usually means dollars lost, whether through loss of services to users or loss of telemetry on what the users are doing. This is when it becomes critical to configure replication within the Kafka cluster (see [Chapter 7](#)), which is also when it is important to consider the physical location of brokers in their racks in the datacenter. A datacenter environment that has a concept of fault zones is preferable. If not addressed prior to deploying Kafka, expensive maintenance to move servers around may be needed.

Kafka can assign new partitions to brokers in a rack-aware manner, making sure that replicas for a single partition do not share a rack. To do this, the `broker.rack` configuration for each broker must be set properly. This config can be set to the fault domain in cloud environments as well for similar reasons. However, this only applies to partitions that are newly created. The Kafka cluster does not monitor for partitions that are no longer rack aware (for example, as a result of a partition reassignment), nor does it automatically correct this situation. It is recommend to use tools that keep your cluster balanced properly to maintain rack awareness, such as Cruise Control (see [Appendix B](#)). Configuring this properly will help to ensure continued rack awareness over time.

Overall, the best practice is to have each Kafka broker in a cluster installed in a different rack, or at the very least not share single points of failure for infrastructure services such as power and network. This typically means at least deploying the servers that will run brokers with dual power connections (to two different circuits) and dual network switches (with a bonded interface on the servers themselves to failover seamlessly). Even with dual connections, there is a benefit to having brokers in completely separate racks. From time to time, it may be necessary to perform physical maintenance on a rack or cabinet that requires it to be offline (such as moving servers around or rewiring power connections).

Colocating Applications on ZooKeeper

Kafka utilizes ZooKeeper for storing metadata information about the brokers, topics, and partitions. Writes to ZooKeeper are only performed on changes to the membership of consumer groups or on changes to the Kafka cluster itself. This amount of traffic is generally minimal, and it does not justify the use of a dedicated ZooKeeper ensemble for a single Kafka cluster. In fact, many deployments will use a single ZooKeeper ensemble for multiple Kafka clusters (using a chroot ZooKeeper path for each cluster, as described earlier in this chapter).

As time goes on, dependency on ZooKeeper is shrinking. In version 2.8.0, Kafka is introducing an early-access look at a completely ZooKeeper-less Kafka, but it is still not production ready. However, we can still see this reduced reliance on ZooKeeper in versions leading up to this. For example, in older versions of Kafka, consumers (in addition to the brokers) utilized ZooKeeper to directly store information about the composition of the consumer group and what topics it was consuming, and to periodically commit offsets for each partition being consumed (to enable failover between consumers in the group). With version 0.9.0.0, the consumer interface was changed, allowing this to be managed directly with the Kafka brokers. In each 2.x release of Kafka, we see additional steps to removing ZooKeeper from other required paths of Kafka. Administration tools now connect directly to the cluster and have deprecated the need to connect to ZooKeeper directly for operations such as topic creations, dynamic configuration changes, etc. As such, many of the command-line tools that previously used the `--zookeeper` flags have been updated to use the `--bootstrap-server` option. The `--zookeeper` options can still be used but have been deprecated and will be removed in the future when Kafka is no longer required to connect to ZooKeeper to create, manage, or consume from topics.

However, there is a concern with consumers and ZooKeeper under certain configurations. While the use of ZooKeeper for such purposes is deprecated, consumers have a configurable choice to use either ZooKeeper or Kafka for committing offsets, and they can also configure the interval between commits. If the consumer uses ZooKeeper for offsets, each consumer will perform a ZooKeeper write at every interval for every partition it consumes. A reasonable interval for offset commits is 1 minute, as this is the period of time over which a consumer group will read duplicate messages in the case of a consumer failure. These commits can be a significant amount of ZooKeeper traffic, especially in a cluster with many consumers, and will need to be taken into account. It may be necessary to use a longer commit interval if the ZooKeeper ensemble is not able to handle the traffic. However, it is recommended that consumers using the latest Kafka libraries use Kafka for committing offsets, removing the dependency on ZooKeeper.

Outside of using a single ensemble for multiple Kafka clusters, it is not recommended to share the ensemble with other applications, if it can be avoided. Kafka is sensitive to ZooKeeper latency and timeouts, and an interruption in communications with the ensemble will cause the brokers to behave unpredictably. This can easily cause multiple brokers to go offline at the same time should they lose ZooKeeper connections, which will result in offline partitions. It also puts stress on the cluster controller, which can show up as subtle errors long after the interruption has passed, such as when trying to perform a controlled shutdown of a broker. Other applications that can put stress on the ZooKeeper ensemble, either through heavy usage or improper operations, should be segregated to their own ensemble.

Summary

In this chapter we learned how to get Apache Kafka up and running. We also covered picking the right hardware for your brokers, and specific

concerns around getting set up in a production environment. Now that you have a Kafka cluster, we will walk through the basics of Kafka client applications. The next two chapters will cover how to create clients for both producing messages to Kafka ([Chapter 3](#)) as well as consuming those messages out again ([Chapter 4](#)).